



GSSS

GEETHA SHISHU
SHIKSHANA SANGHA (R)

**SIMHA SUBBAMAHALAKSHMI
FIRST GRADE COLLEGE**

UNIT – 4

COMBINATIONAL AND SEQUENTIAL LOGIC CIRCUIT

Types of Logic Circuits: There are two types of Digital circuits depending on their output and memory used:

- Combinational circuit** and
- Sequential circuit**

Combinational Circuit

These circuits are developed using AND, OR, NOT, NAND, and NOR logic gates. These logic gates are building blocks of combinational circuits. A combinational circuit consists of input variables and output variables.

Combinational circuits are specially designed using multiple interconnected logic gates such that the output will be generated by computing the logical combinations of the present input only. No clock pulse is present here. Moreover, no previously stored value or state is taken into consideration here. The output is independent of previous states.

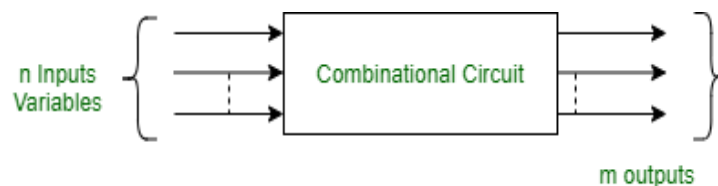


Figure - Block diagram of Combinational circuit

Examples:

In areas combinational circuits are used which are discussed below:

- **Adders and Subtractors:** Combinational circuits are used to perform mathematical operations of binary numbers. Adder and Subtractor uses combinational circuit logic where it takes two or more binary numbers as input and performs addition/subtraction to generate output.
- **Multiplexers and Demultiplexers:** Multiplexer is a special kind of combinational circuit where several number of inputs are present. From that one input can be selected and that will be transmitted as an output based on selection signal. In the other hand, Demultiplexers also select one input signal but transmits it to one of the several outputs.
- **Encoders and Decoders:** Encoders also use combinational circuits to convert a multiple set of input signal to smaller set of output signal but the resultant value is unchanged.
Inversely, Decoders convert a small set of input signal to a large set of output signal without changing the resultant value.
Basically input of Encoder = output of Decoder and vice versa.

Sequential circuits

A sequential circuit is specified by a time sequence of inputs, outputs, and internal states. The output of a sequential circuit depends not only on the combination of present inputs but also on the previous outputs. Unlike combinational circuits, sequential circuits include memory elements with combinational circuits. Some examples are counters and shift registers.

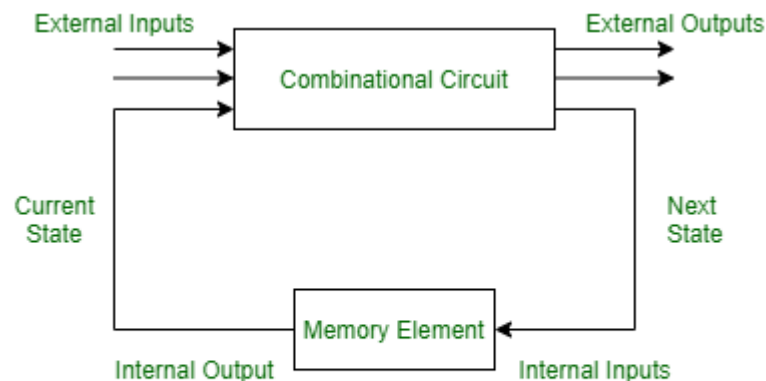


Figure - Sequential Circuit

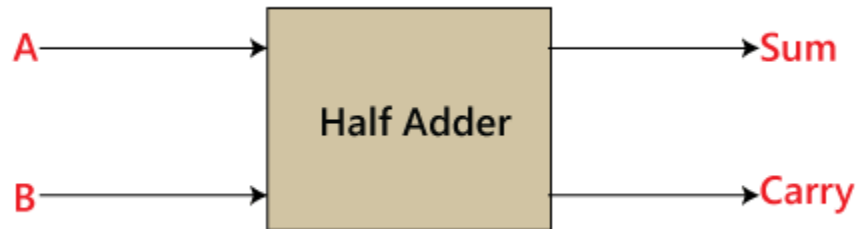
Difference between Combinational and Sequential Circuit

Aspect	Combinational Circuit	Sequential Circuit
Definition	Output depends only on the current inputs.	Output depends on both current inputs and past states (memory).
Memory Elements	Does not require memory elements.	Requires memory elements like flip-flops or latches.
Timing Dependency	Output is immediate, based on input changes.	Output is dependent on clock pulses and previous states.
Clock Signal	No clock signal required.	Requires a clock signal to synchronize state changes.
Design Complexity	Simpler design without the need for memory.	More complex due to memory and clock management.
Speed	Faster, as outputs change instantly with inputs.	Slower due to dependency on clock cycles and past states.
Functionality	Performs basic logical operations without sequence dependency.	Performs operations that require sequences or timed events.
Examples	Adders, Subtractors, Multiplexers, Encoders.	Counters, Shift Registers, Flip-Flops, State Machines.
Power Consumption	Generally lower power consumption.	Higher power consumption due to memory and clock circuitry.
Application	Used in tasks requiring direct logical operations (e.g., arithmetic).	Used in applications involving sequential operations (e.g., counters, registers).

Combinational circuit

Half Adder

Half Adder is a combinational logic circuit that is designed by connecting one EX- OR gate and one AND gate. The half-adder circuit has two inputs: A and B, which add two input digits and generate a carry and a sum.



The output obtained from the EX-OR gate is the sum of the two numbers while that obtained by AND gate is the carry. There will be no forwarding of carry addition because there is no logic gate to process that. Thus, this is called the Half Adder circuit.

Truth Table of Half Adder

The Truth Table for Half Added is Given as

Truth Table			
Input		Output	
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

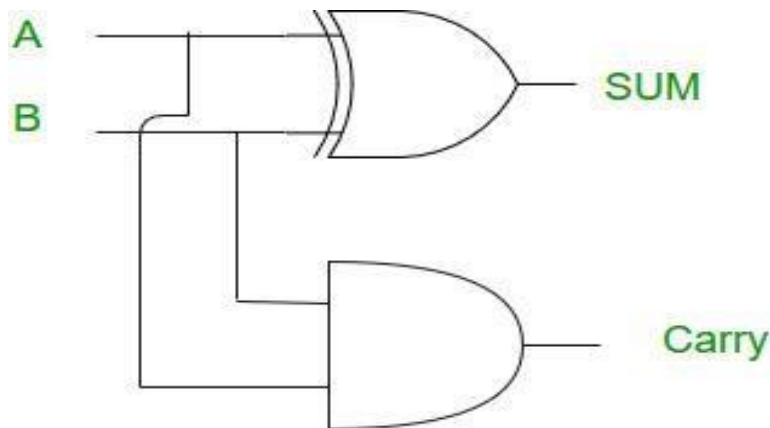
Logical Expression

The Logical Expression for half added is given as

$$\text{Sum} = A \oplus B$$

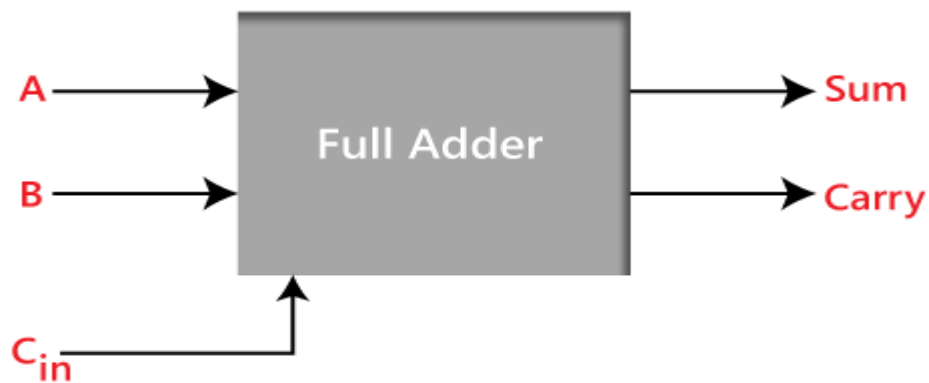
$$\text{Carry} = A \text{ AND } B$$

Logic Circuit



Full adder

Full Adder is the adder that adds three inputs and produces two outputs. The first two inputs are A and B and the third input is an input carry as C-IN. The output carry is designated as C-OUT and the normal output is designated as S which is SUM.



Truth Table of Full Adder

Input			Output	
A	B	C _{in}	Sum	C _{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Logical Expression

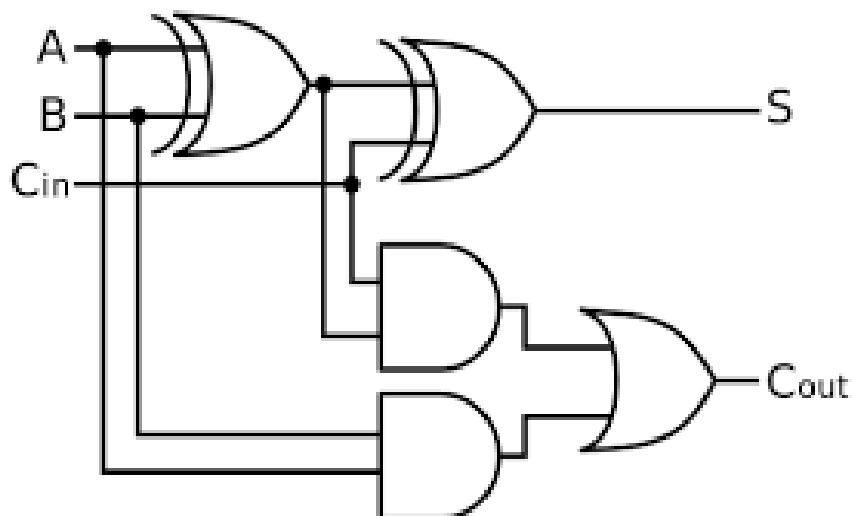
The sum (S) of the full-adder is the XOR of A, B, and C_{in}. Therefore,

$$\text{Sum}, S = A \oplus B \oplus C_{in}$$

The carry (C) of the half-adder is the AND of A and B. Therefore,

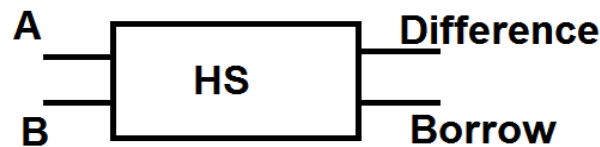
$$\text{Carry}, C = AB + AC_{in} + BC_{in}$$

Logic Circuit



Half Subtractor

Half subtractor is a combination circuit with two inputs and two outputs that are different and borrow. It produces the difference between the two binary bits at the input and also produces an output (Borrow) to indicate if a 1 has been borrowed. In the subtraction (A-B), A is called a Minuend bit and B is called a Subtrahend bit.



Half Subtractor

Truth Table

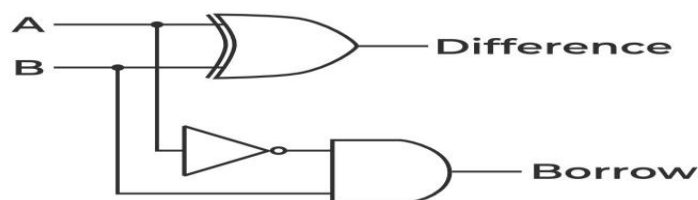
A	B	Diff	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Logical Expression The SOP form of the Diff and Borrow is as follows:

$$\text{Diff} = A \oplus B$$

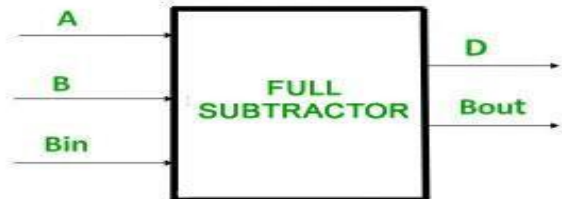
$$\text{Borrow} = A'B$$

Implementation



Full Subtractor

A full subtractor is a combinational circuit that performs subtraction of two bits, one is minuend and other is subtrahend, taking into account borrow of the previous adjacent lower minuend bit. This circuit has three inputs and two outputs. The three inputs A, B and Bin, denote the minuend, subtrahend, and previous borrow, respectively. The two outputs, D and Bout represent the difference and output borrow, respectively.



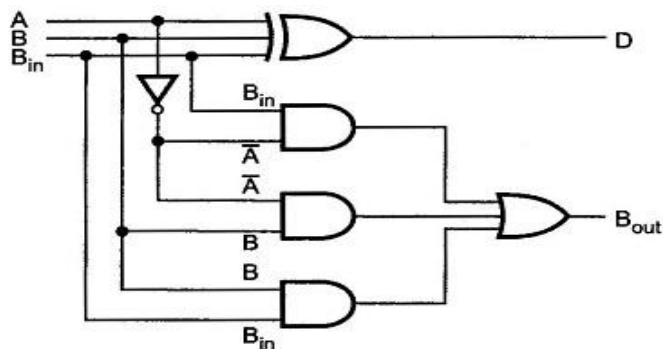
Truth Table

Inputs			Outputs	
A	B	Borrow _{in}	Diff	Borrow
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Logical Expression The SOP form of the Diff and Borrow is as follows:

$$\text{Diff} = A \oplus B \oplus B_{in}$$

$$\text{Borrow} = A'B_{in} + A'B + BB_{in}$$



FLIP FLOPS

- A flip-flop is a small digital circuit that has two stable states (0 or 1).
- It is used to store one bit of data.
- By giving it the right inputs, we can change or hold the stored value.

Types of Flip-Flops

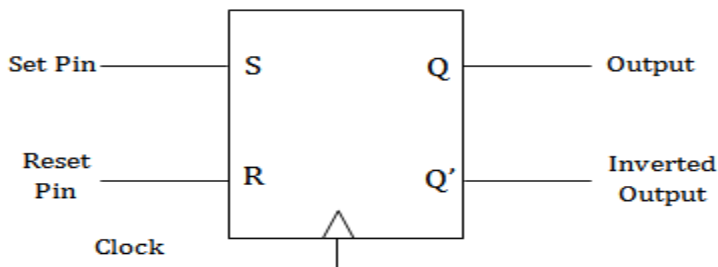
- S-R Flip-Flop
- J-K Flip-Flop
- T Flip-Flop

S-R FLIP FLOPS

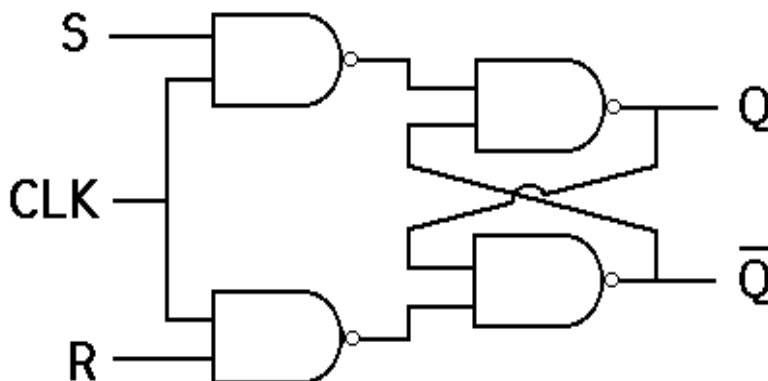
The SR flip-flop is the most common and simple type.

- It has two inputs: **Set (S)** and **Reset (R)**.
- If $S = 1$, the output $Q = 1$ and $Q' = 0$.
- If $R = 1$, the output $Q = 0$ and $Q' = 1$.

Symbol



Circuit Diagram



Truth table

S	R	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	X

Characteristic equation

$$Q_{n+1} = S + R'Q_n$$

J-K FLIPFLOPS

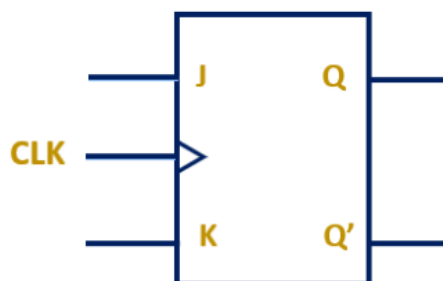
The JK flip-flop is an improved version of the SR flip-flop.

- It removes the problem of the undefined state when both inputs are 1.
- It has two inputs: J and K.

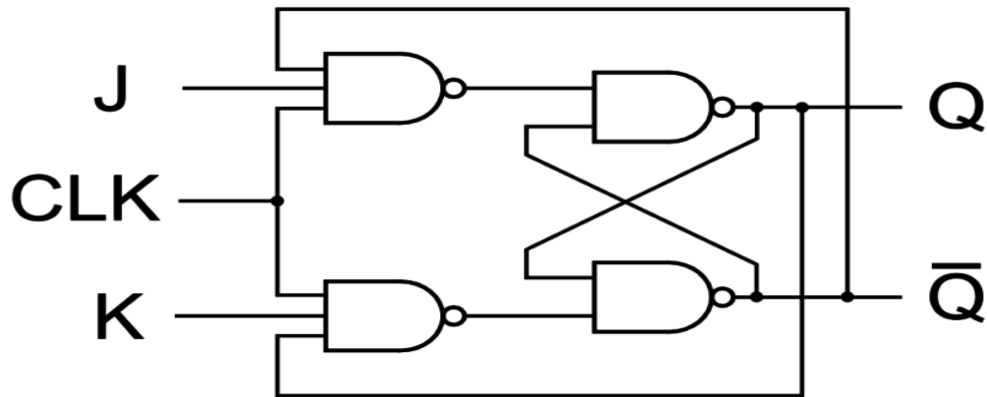
The rules are:

- $J = 0, K = 0 \rightarrow$ No change in output.
- $J = 1, K = 0 \rightarrow$ Output is Set ($Q = 1$).
- $J = 0, K = 1 \rightarrow$ Output is Reset ($Q = 0$).
- $J = 1, K = 1 \rightarrow$ Output toggles (changes from 0 to 1 or 1 to 0).

Symbol



Circuit Diagram



Truth Table

J	K	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	Q_n'

Characteristic equation

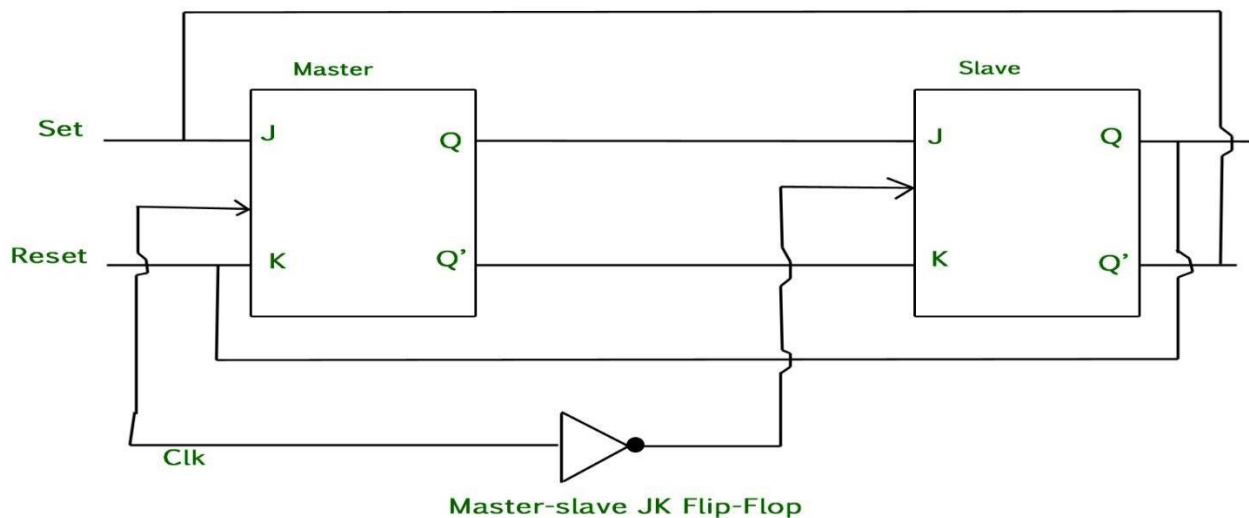
$$Q_{n+1} = J Q_n' + K' Q_n$$

Master-Slave JK Flip Flop

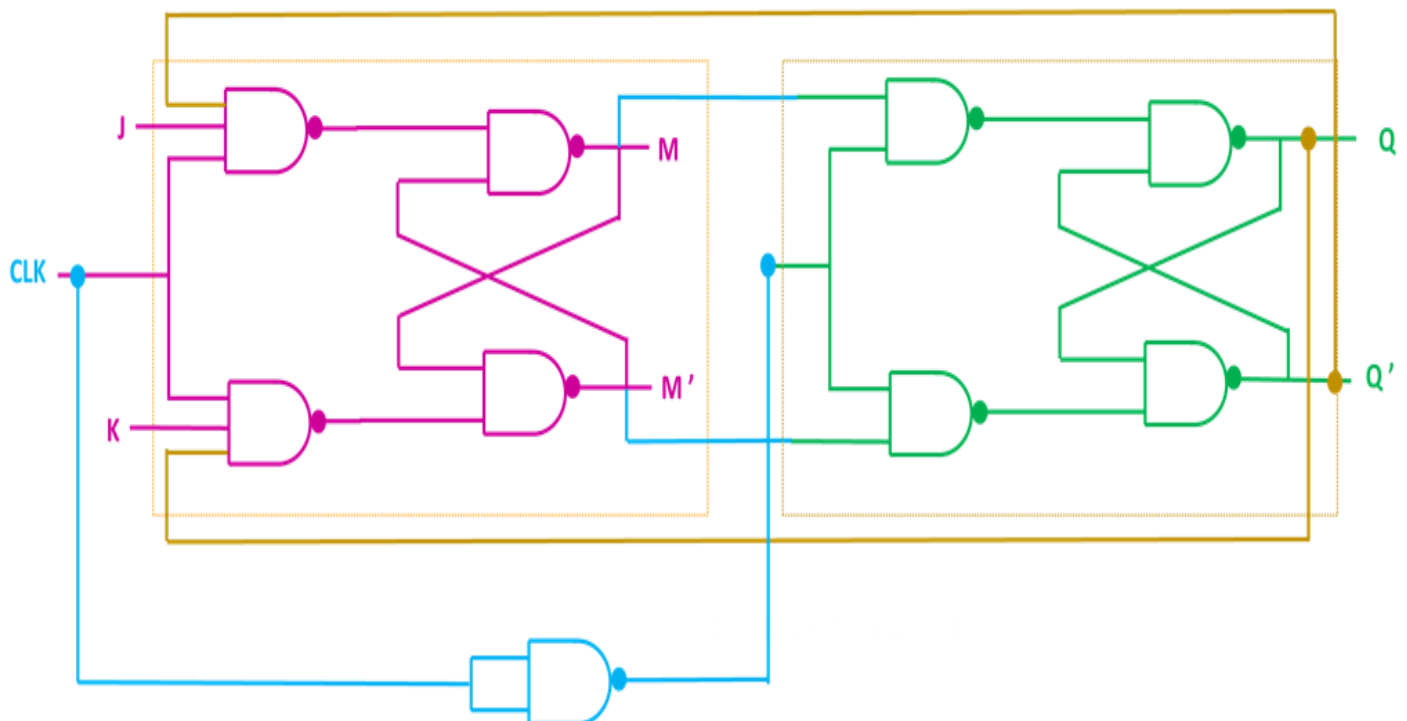
In a **JK flip-flop**, if both inputs (**J = 1** and **K = 1**) and the **clock (CLK = 1)** stay high for a long time, the output **Q** **keeps toggling** again and again. This creates an **uncertain output**, called the **race-around condition**.

To avoid this, the clock signal is kept **high only for a very short time** so the flip-flop doesn't toggle repeatedly

Symbol



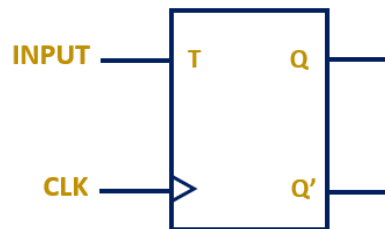
Circuit Diagram



T FLIP FLOPS

T flip flop is similar to JK flip flop. Just tie both J and K inputs together to get a T Flip flop. Just like the D flip flop, it has only one external input along with a clock.

Symbol



Truth Table

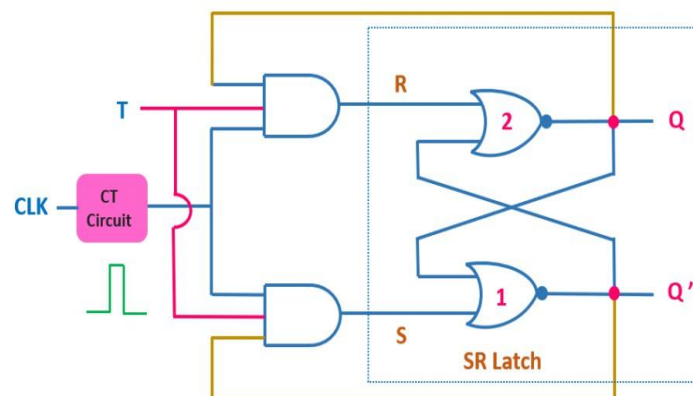
CLK	T	Q_{n+1}
↑	0	Q_n
↑	1	Q_n'

Characteristic Equation $Q_{n+1} = T Q_n' + T' Q_n$
 $= T \oplus Q_n$

Excitation Table

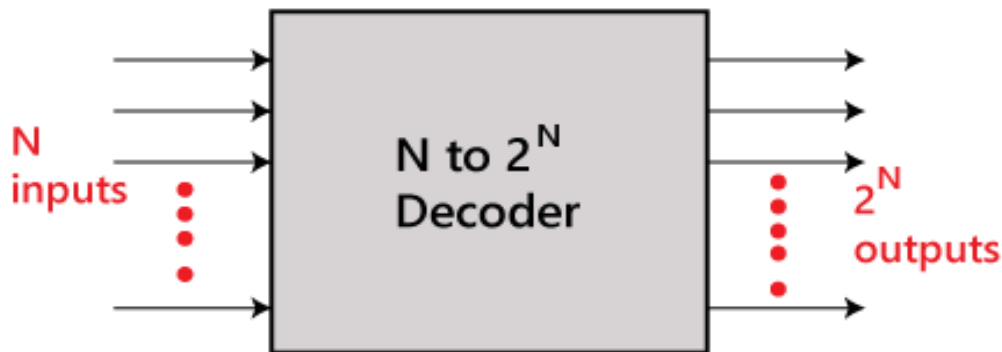
Q_n	Q_{n+1}	T
0 → 0	0	0
1 → 0	0	1
0 → 1	1	1
1 → 1	1	0

Circuit Diagram



DECODERS

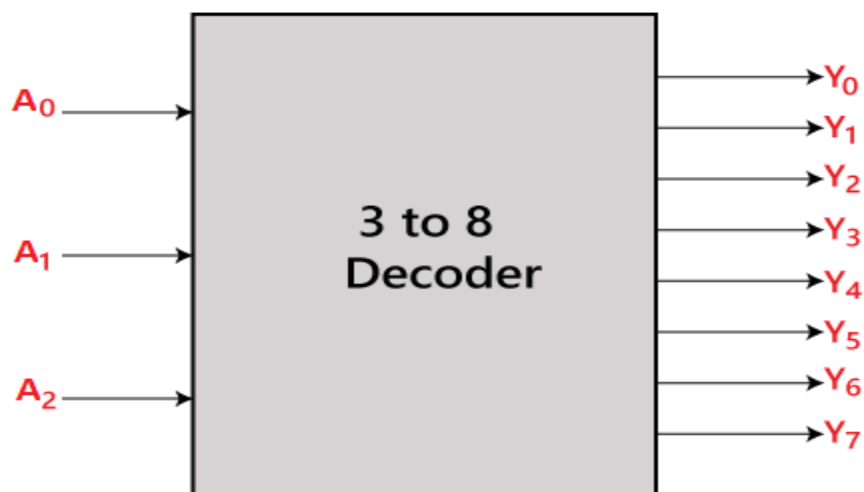
The combinational circuit that change the binary information into 2^N output lines is known as Decoders. The binary information is passed in the form of N input lines. The output lines define the 2^N -bit code for the binary information. In simple words, the Decoder performs the reverse operation of the Encoder. At a time, only one input line is activated for simplicity. The produced 2^N -bit output code is equivalent to the binary information.



3 TO 8 LINE DECODER

The 3 to 8 line decoder is also known as Binary to Octal Decoder. In a 3 to 8 line decoder, there is a total of eight outputs, i.e., $Y_0, Y_1, Y_2, Y_3, Y_4, Y_5, Y_6$, and Y_7 and three inputs, i.e., A_0, A_1 , and A_2 . This circuit has an enable input 'E'. Just like 2 to 4 line decoder, when enable 'E' is set to 1, one of these four outputs will be 1. The block diagram and the truth table of the 3 to 8 line encoder are given below.

Block Diagram



Truth Table

Enable	INPUTS			Outputs							
E	A ₂	A ₁	A ₀	Y ₇	Y ₆	Y ₅	Y ₄	Y ₃	Y ₂	Y ₁	Y ₀
0	x	x	x	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1
1	0	0	1	0	0	0	0	0	0	1	0
1	0	1	0	0	0	0	0	0	1	0	0
1	0	1	1	0	0	0	0	1	0	0	0
1	1	0	0	0	0	0	1	0	0	0	0
1	1	0	1	0	0	1	0	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0

The logical expression of the term Y₀, Y₁, Y₂, Y₃, Y₄, Y₅, Y₆, and Y₇ is as follows:

$$Y_0 = A_0' . A_1' . A_2'$$

$$Y_1 = A_0 . A_1' . A_2'$$

$$Y_2 = A_0' . A_1 . A_2'$$

$$Y_3 = A_0 . A_1 . A_2'$$

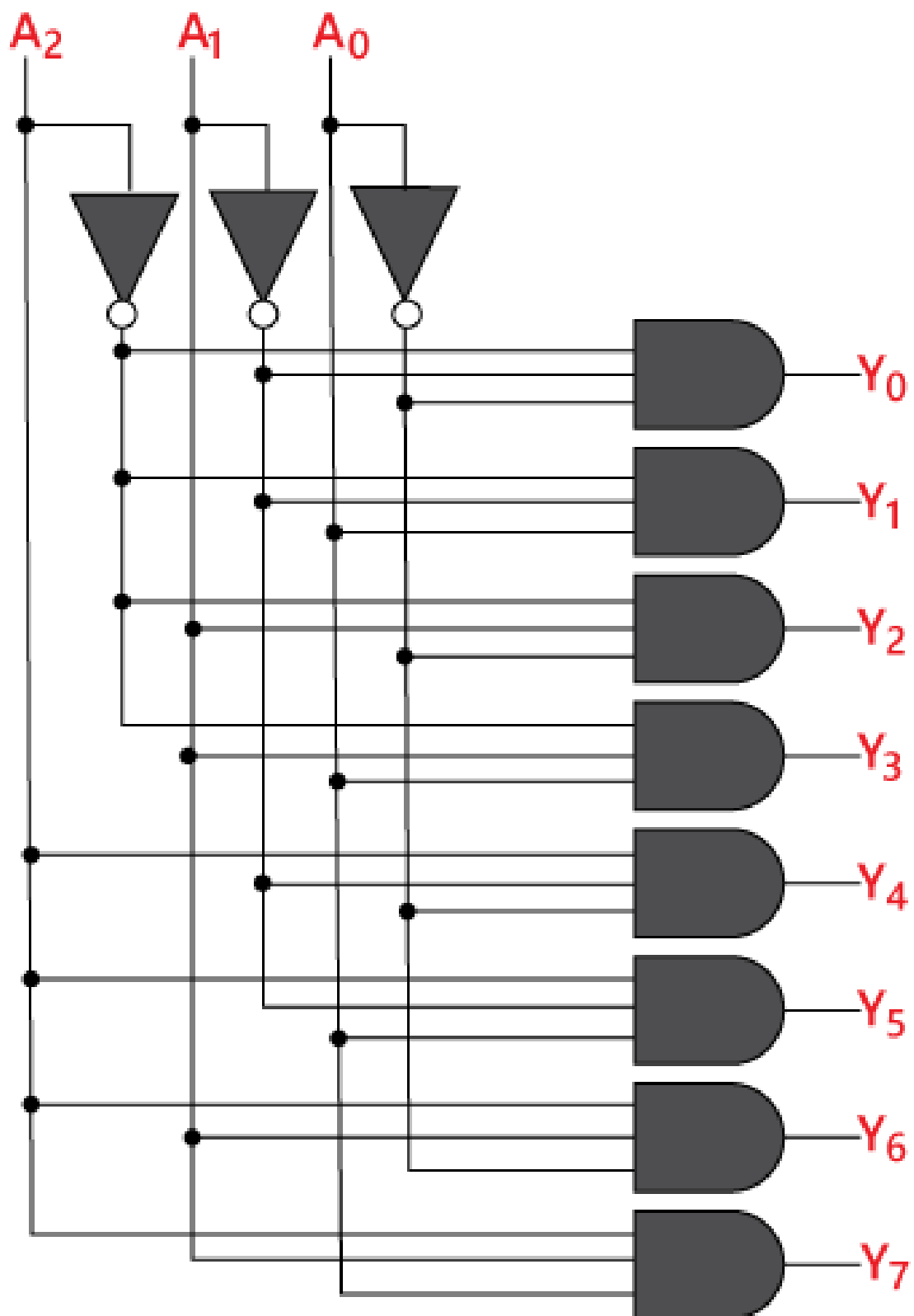
$$Y_4 = A_0' . A_1' . A_2$$

$$Y_5 = A_0 . A_1' . A_2$$

$$Y_6 = A_0' . A_1 . A_2$$

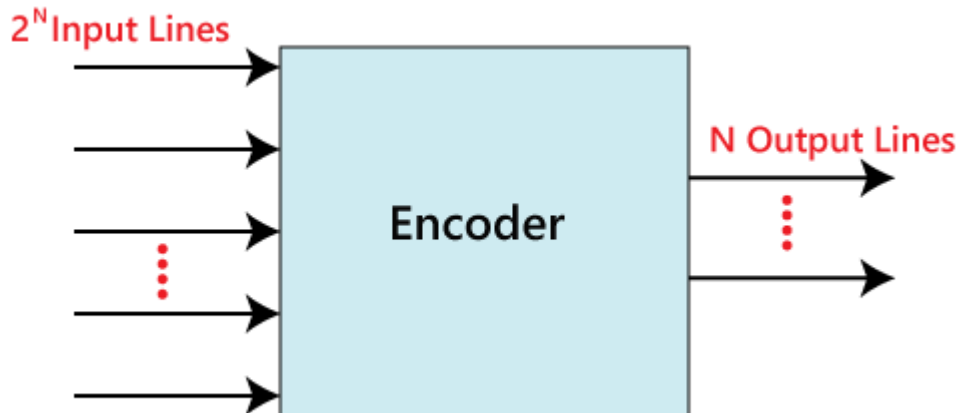
$$Y_7 = A_0 . A_1 . A_2$$

Logical circuit of the above expressions



ENCODERS

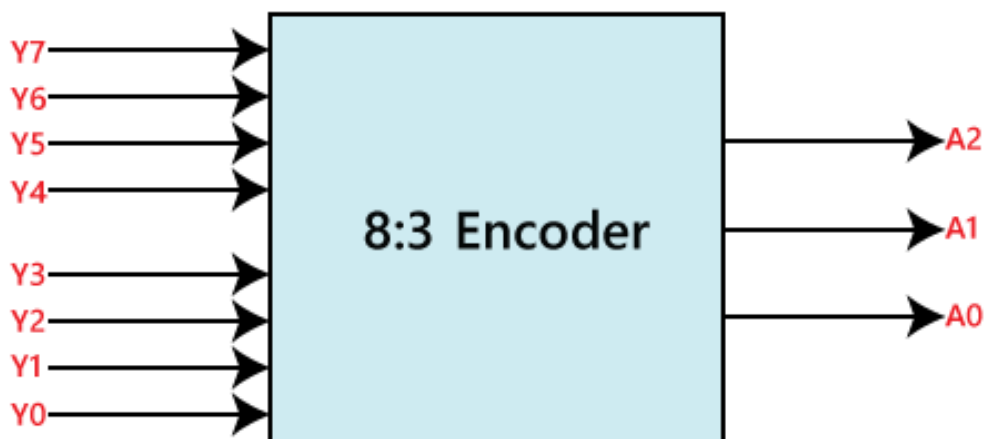
The combinational circuits that change the binary information into N output lines are known as Encoders. The binary information is passed in the form of 2^N input lines. The output lines define the N -bit code for the binary information. In simple words, the Encoder performs the reverse operation of the Decoder. At a time, only one input line is activated for simplicity. The produced N -bit output code is equivalent to the binary information.



8 to 3 line Encoder / Octal to Binary Encoder

The 8 to 3 line Encoder is also known as **Octal to Binary Encoder**. In 8 to 3 line encoder, there is a total of eight inputs, i.e., $Y_0, Y_1, Y_2, Y_3, Y_4, Y_5, Y_6$, and Y_7 and three outputs, i.e., A_0, A_1 , and A_2 . In 8-input lines, one input-line is set to true at a time to get the respective binary code in the output side. Below are the block diagram and the truth table of the 8 to 3 line encoder.

Block Diagram



Truth Table

INPUTS								OUTPUTS		
Y ₇	Y ₆	Y ₅	Y ₄	Y ₃	Y ₂	Y ₁	Y ₀	A ₂	A ₁	A ₀
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

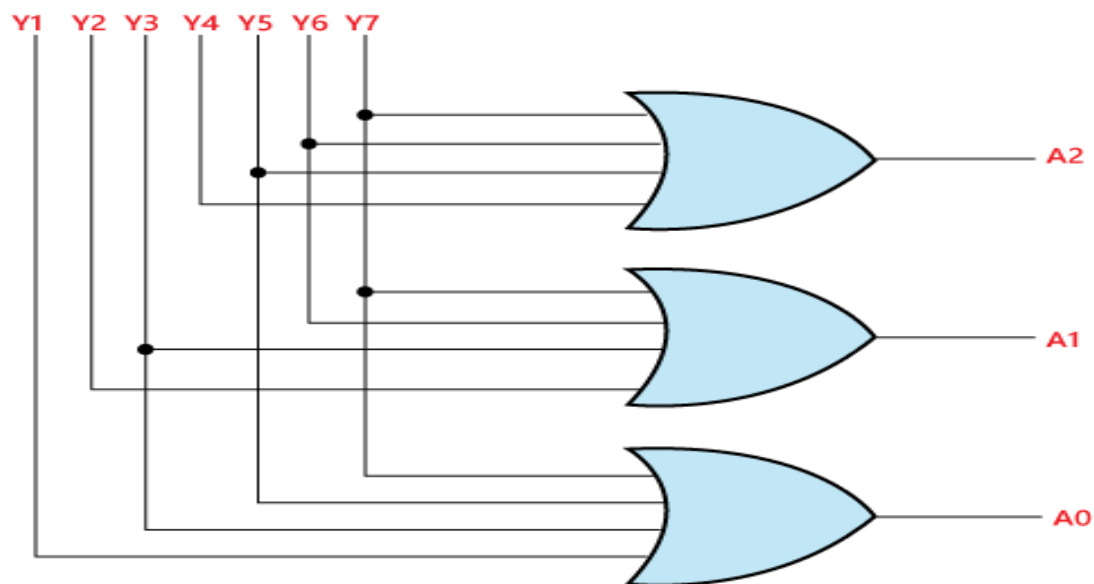
The **logical expression** of the term A0, A1, and A2 are as follows:

$$A_2 = Y_4 + Y_5 + Y_6 + Y_7$$

$$A_1 = Y_2 + Y_3 + Y_6 + Y_7$$

$$A_0 = Y_7 + Y_5 + Y_3 + Y_1$$

Logical circuit of the above expressions



Multiplexer

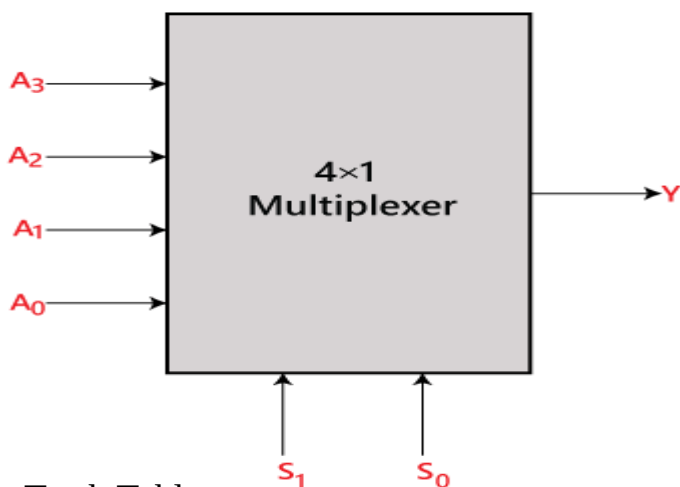
A multiplexer is a combinational circuit that has 2^n input lines and a single output line. Simply, the multiplexer is a multi-input and single-output combinational circuit. The binary information is received from the input lines and directed to the output line. On the basis of the values of the selection lines, one of these data inputs will be connected to the output.

Unlike encoder and decoder, there are n selection lines and 2^n input lines. So, there is a total of 2^N possible combinations of inputs. A multiplexer is also treated as Mux.

4×1 Multiplexer / 4 to 1-line Mux

In the 4×1 multiplexer, there is a total of four inputs, i.e., A_0 , A_1 , A_2 , and A_3 , 2 selection lines, i.e., S_0 and S_1 and single output, i.e., Y . On the basis of the combination of inputs that are present at the selection lines S_0 and S_1 , one of these 4 inputs are connected to the output. The block diagram and the truth table of the 4×1 multiplexer are given below.

Block Diagram:



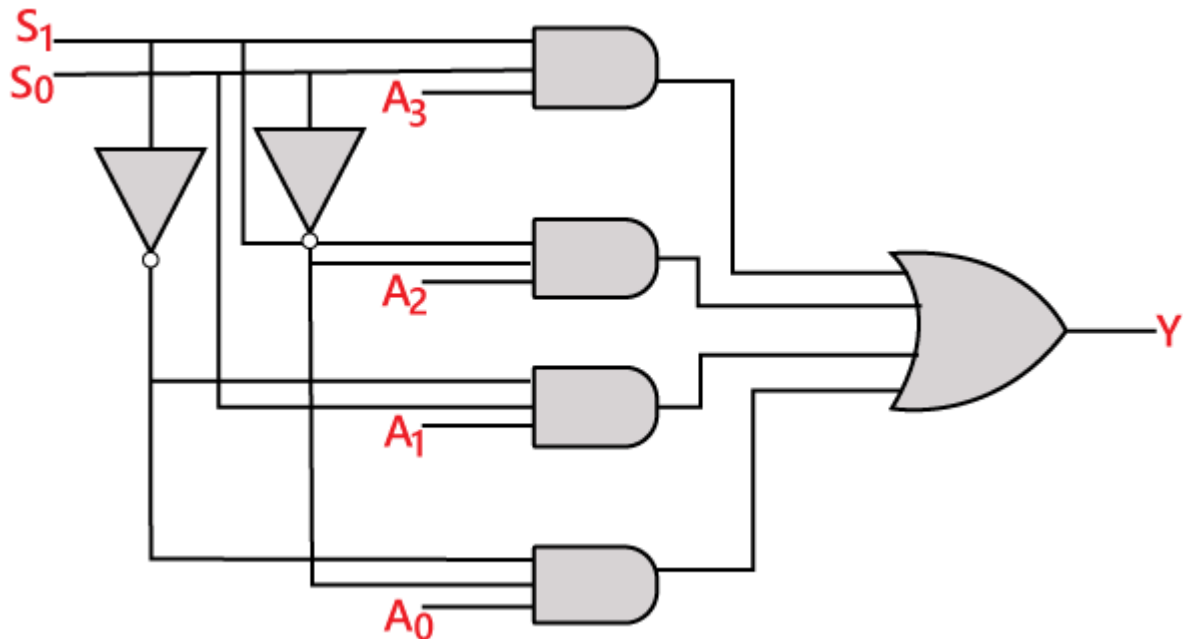
Truth Table:

INPUTS		Output
S_1	S_0	Y
0	0	A_0
0	1	A_1
1	0	A_2
1	1	A_3

The **logical expression** of the term Y is as follows:

$$Y = S_1' S_0' A_0 + S_1' S_0 A_1 + S_1 S_0' A_2 + S_1 S_0 A_3$$

Logical circuit of the above expression is given below:



COUNTERS

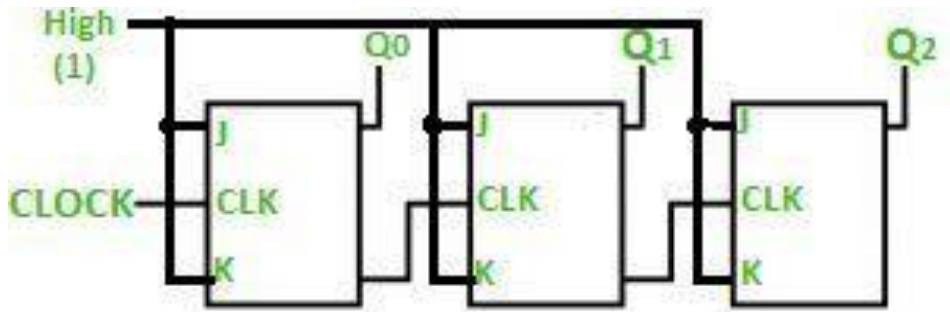
A special type of sequential circuit used to count the pulse is known as a counter, or a collection of flip flops where the clock signal is applied is known as counters.

The counter is one of the widest applications of the flip flop. Based on the clock pulse, the output of the counter contains a predefined state. The number of the pulse can be counted using the output of the counter.

3 Bits Binary Ripple Counter

Ripple counter is a cascaded arrangement of flip-flops where the output of one flip-flop drives the clock input of the following flip-flop. The number of flip flops in the cascaded arrangement depends upon the number of different logic states that it goes through before it repeats the sequence a parameter known as the modulus of the counter.

3-bit Ripple counter using a JK flip-flop



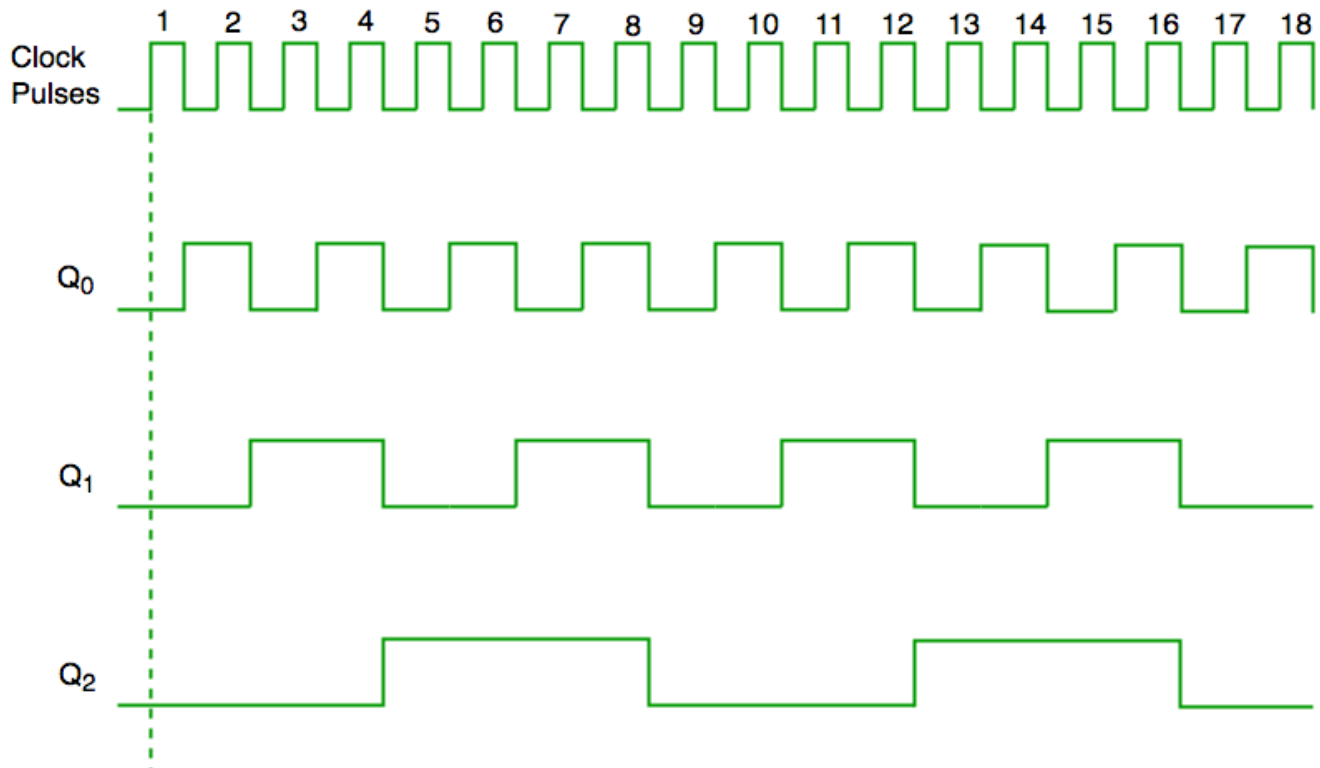
In the circuit shown in the above figure, Q0(LSB) will toggle for every clock pulse because JK flip-flop works in toggle mode when both J and K are applied 1, 1, or high input. The following counter will toggle when the previous one changes from 1 to 0.

Truth Table is as follows:

Counter State	Q_2	Q_1	Q_0
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

Timing diagram

Let us assume that the clock is negative edge triggered so the above the counter will act as an up counter because the clock is negative edge triggered and output is taken from Q.



Counters are used very frequently to divide clock frequencies and their uses mainly involve digital clocks and in multiplexing. The widely known example of the counter is parallel to serial data conversion logic.

Advantages of Ripple Counter in Digital Logic

- Can be easily designed by T flip-flop or [D flip-flop](#).
- Can be used in low speed circuits & divide by n-counters.
- Used as Truncated counters to design any mode number counters (i.e. Mod 4, Mod 3)

Disadvantages of Ripple Counter in Digital Logic

- Extra flip-flop are needed to do resynchronization.
- To count the sequence of truncated counters, additional feedback logic is needed.
- Propagation delay of asynchronous counters is very large, while counting the large number of bits.
- Counting errors may occur due to propagation delay for high clock frequencies.

SHIFT REGISTER

Shift Registers are used to implement **Arithmetic operations**.

Flip flops can be used to store a single bit of binary data (1 or 0). However, in order to store multiple bits of data, we need multiple flip-flops. N flip flops are to be connected in order to store n bits of data.

A **Register** is a device that is used to store such information. It is a group of flip-flops connected in series used to store multiple bits of data. The information stored within these registers can be transferred with the help of **shift registers**.

Shift Register is a group of flip flops used to store multiple bits of data. The bits stored in such registers can be made to move within the registers and in/out of the registers by applying clock pulses.

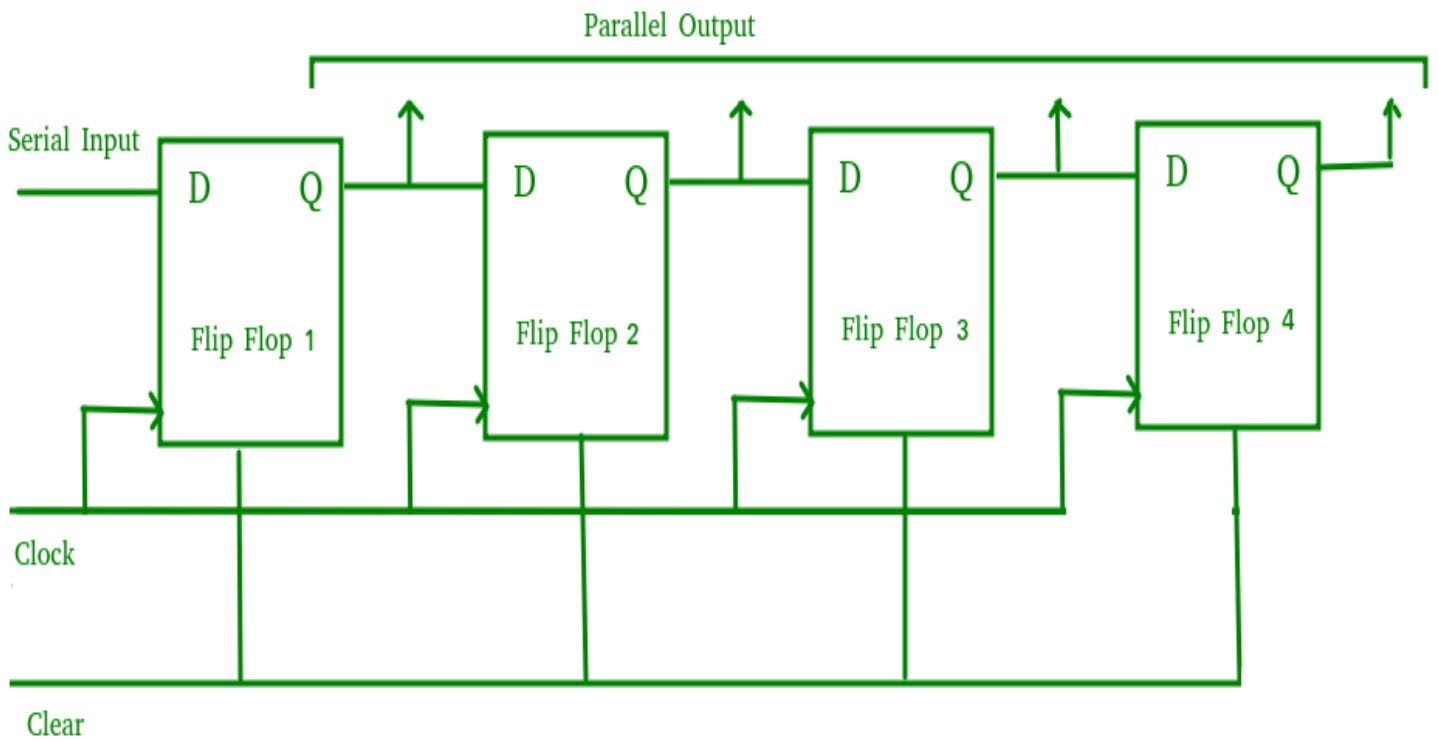
An n-bit shift register can be formed by connecting n flip-flops where each flip-flop stores a single bit of data. The registers which will shift the bits to the left are called “**Shift left registers**”. The registers which will shift the bits to the right are called “**Shift right registers**”. Shift registers are basically of following types.

Types of Shift Registers

1. **Serial In Serial Out(SISO) shift register**
2. **Serial In parallel Out (SIPO)shift register**
3. **Parallel In Serial Out(PISO) shift register**
4. **Parallel In parallel Out (PIPO) shift register**

Serial In parallel Out (SIPO)shift register

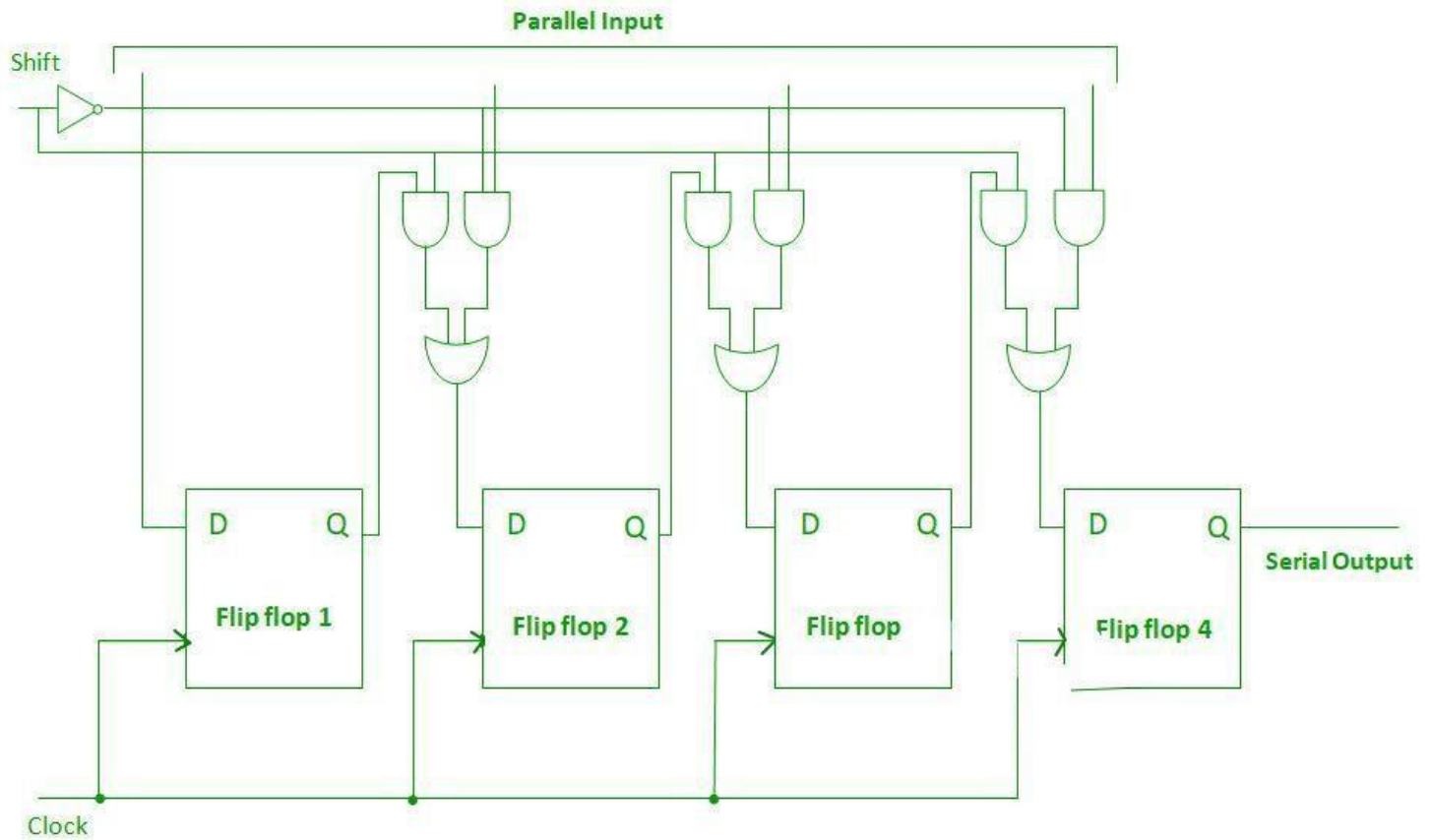
The shift register, which allows serial input (one bit after the other through a single data line) and produces a parallel output is known as the Serial-In Parallel-Out shift register. The logic circuit given below shows a serial-in-parallel-out shift register. The circuit consists of four D flip-flops which are connected. The clear (CLR) signal is connected in addition to the clock signal to all 4 flip flops in order to RESET them. The output of the first flip-flop is connected to the input of the next flip flop and so on. All these flip-flops are synchronous with each other since the same clock signal is applied to each flip-flop.



The above circuit is an example of a shift right register, taking the serial data input from the left side of the flip-flop and producing a parallel output. They are used in communication lines where demultiplexing of a data line into several parallel lines is required because the main use of the SIPO register is to convert serial data into parallel data.

Parallel-In Serial-Out Shift Register (PISO)

The shift register, which allows parallel input (data is given separately to each flip flop and in a simultaneous manner) and produces a serial output is known as a Parallel-In Serial-Out shift register. The logic circuit given below shows a parallel-in-serial-out shift register. The circuit consists of four D flip-flops which are connected. The clock input is directly connected to all the flip-flops but the input data is connected individually to each flip-flop through a [multiplexer](#) at the input of every flip-flop. The output of the previous flip-flop and parallel data input are connected to the input of the MUX and the output of MUX is connected to the next flip-flop. All these flip-flops are synchronous with each other since the same clock signal is applied to each flip-flop.



Thank You
All the Best for your Exam
All the Best for your Exam